

# Руководство разработчика и развёртывания

Техническая эксплуатация системы

|           |                                             |
|-----------|---------------------------------------------|
| Версия    | 1.0                                         |
| Дата      | 22 июля 2026 г.                             |
| Основание | Актуальная структура исходного кода проекта |

## Содержание

- 1. Архитектура
- 2. Состав репозитория
- 3. Требования и конфигурация
- 4. Развёртывание Docker Compose
- 5. Локальная разработка
- 6. База данных и миграции
- 7. API и фоновые задачи
- 8. Проверка, мониторинг и резервное копирование

## 1. Архитектура

Система построена как web-приложение: React SPA обслуживается Nginx, обращается к FastAPI по HTTP API; API хранит данные в PostgreSQL, отправляет длительные расчёты в Celery, а Redis используется как брокер и backend задач. Планировщик использует Google OR-Tools CP-SAT.

| Компонент | Технологии                                       | Функция                                                                          |
|-----------|--------------------------------------------------|----------------------------------------------------------------------------------|
| Frontend  | React 19, TypeScript, Vite, Zustand, TailwindCSS | Операторский интерфейс: импорт, справочники, календарь, расчёты, сетка и журнал. |
| Backend   | FastAPI, Pydantic v2, SQLAlchemy 2.0 async       | REST API, бизнес-правила, валидация конфликтов, экспорт Excel.                   |
| Database  | PostgreSQL 17                                    | Транзакционное хранение справочников, расчётов, версий и расписания.             |

| Компонент | Технологии        | Функция                                                                |
|-----------|-------------------|------------------------------------------------------------------------|
| Worker    | Celery 5, Redis 8 | Асинхронный запуск и контроль ресурсоёмких задач генерации.            |
| Optimizer | Google OR-Tools   | Построение и решение модели расписания с ограничениями и приоритетами. |

## 2. Состав репозитория

| Путь               | Содержимое                                                                       |
|--------------------|----------------------------------------------------------------------------------|
| src/app/api/routes | FastAPI endpoints: scheduler, planning, reference.                               |
| src/app/services   | Бизнес-логика импорта, генерации, жизненного цикла, проверки качества, экспорта. |
| src/app/database   | SQLAlchemy-модели и асинхронная сессия БД.                                       |
| src/app/migrations | Alembic-конфигурация и версии миграций.                                          |
| src/app/tests      | Тесты здоровья, планирования, справочников, генерации и lifecycle.               |
| spa/src/pages      | Страницы интерфейса оператора.                                                   |
| spa/src/store      | Общее состояние интерфейса на Zustand.                                           |
| docker-compose.yml | Production-подобная композиция сервисов.                                         |

## 3. Требования и конфигурация

Для контейнерного запуска необходимы Docker Engine с Docker Compose. Для локальной разработки: Python 3.10+, Node.js, PostgreSQL и Redis. Производственная БД — PostgreSQL; пароль и секреты не должны храниться в Git.

| Переменная                | Назначение                     | Примечание                                      |
|---------------------------|--------------------------------|-------------------------------------------------|
| POSTGRES_USER             | Пользователь PostgreSQL        | Обязательна для Compose.                        |
| POSTGRES_PASSWORD         | Пароль PostgreSQL              | Используйте уникальный секрет production.       |
| POSTGRES_DATABASE         | Имя БД                         | Используется сервисами backend и worker.        |
| POSTGRES_PORT             | Внешний порт PostgreSQL        | По умолчанию в Compose: 5446.                   |
| SECRET_KEY                | Подпись токенов                | Длинное случайное значение; не публиковать.     |
| CORS_ORIGINS              | Разрешённые источники браузера | Список через запятую; задайте домен production. |
| CELERY_WORKER_CONCURRENCY | Параллелизм worker             | Подбирается по CPU/RAM сервера.                 |

| Переменная               | Назначение             | Примечание                                          |
|--------------------------|------------------------|-----------------------------------------------------|
| DATABASE_URL / REDIS_URL | Подключения приложения | В Compose формируются для внутренних имён сервисов. |
| VITE_API_BASE_URL        | Префикс API SPA        | Задаётся при сборке frontend.                       |

## 4. Развёртывание Docker Compose

1. Скопируйте .env.example в .env и замените значения POSTGRES\_PASSWORD и SECRET\_KEY на безопасные. Проверьте CORS\_ORIGINS для внешнего домена.
2. Соберите и запустите сервисы командой: `docker compose --env-file .env up -d --build`.
3. Проверьте состояние: `docker compose ps`. Backend готов, когда отвечает /health/live и /health/ready; Compose ожидает readiness БД перед запуском API.
4. Откройте frontend на опубликованном порту 80. API доступен через frontend/Nginx либо напрямую на порту 8000, если он не закрыт сетевыми правилами.
5. При первом запуске backend применяет миграции, потому что RUN\_MIGRATIONS=true. Worker запускается после готовности backend и не выполняет миграции.

| Сервис   | Порт/том                                 | Назначение                                    |
|----------|------------------------------------------|-----------------------------------------------|
| postgres | \${POSTGRES_PORT}:5432;<br>postgres_data | Персистентная PostgreSQL 17.                  |
| redis    | redis_data                               | Очередь и результат Celery; AOF включён.      |
| backend  | 8000:8000; uploads, backend_logs         | FastAPI и автоматическое применение миграций. |
| worker   | uploads, backend_logs                    | Celery worker, concurrency configurable.      |
| frontend | 80:80                                    | Сборка SPA и reverse proxy.                   |

**Production.** Разрешите наружу только необходимые порты, завершайте TLS на reverse проху/балансировщике, используйте отдельные секреты, ограничьте доступ к PostgreSQL и регулярно проверяйте журналы backend/worker.

## 5. Локальная разработка

Backend запускается из каталога src. Перед началом создайте виртуальное окружение, установите зависимости из requirements.txt и настройте переменные окружения. SPA запускается из каталога spa после установки зависимостей npm.

| Действие                       | Команда                                                                  |
|--------------------------------|--------------------------------------------------------------------------|
| Установить backend-зависимости | <code>cd src &amp;&amp; python -m pip install -r requirements.txt</code> |

| Действие                   | Команда                                                                              |
|----------------------------|--------------------------------------------------------------------------------------|
| Применить миграции         | <code>cd src &amp;&amp; alembic upgrade head</code>                                  |
| Запустить API              | <code>cd src &amp;&amp; uvicorn main:app --app-dir app --reload</code>               |
| Установить SPA-зависимости | <code>cd spa &amp;&amp; npm install</code>                                           |
| Запустить SPA              | <code>cd spa &amp;&amp; npm run dev</code>                                           |
| Запустить worker           | <code>cd src &amp;&amp; celery --app=worker.celery_app worker --loglevel=INFO</code> |

## 6. База данных и миграции

- Миграции находятся в `src/app/migrations/versions`. Создавайте их из каталога `src: alembic revision --autogenerate -m "описание"`.
- Перед production-миграцией создайте и проверьте резервную копию PostgreSQL; затем примените `alembic upgrade head`.
- Не используйте `AUTO_CREATE_TABLES` как механизм production-эволюции схемы. Эволюция выполняется только миграциями Alembic.
- После изменения модели проверьте `alembic check` и сценарии чтения/записи, затронутые миграцией.

## 7. API и фоновые задачи

Все маршруты прикладного API размещены под `/api/v1`. Подмаршруты: `/scheduler` — импорт, генерация, версии, запись расписания и экспорт; `/planning` — периоды, недели и нагрузка; `/reference` — справочники, доступность и профили правил.

| Группа          | Примеры операций                                                                                                                                                   |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Импорт и данные | <code>POST /scheduler/import/streams</code> ; <code>GET /scheduler/import/history</code> ; <code>PATCH /scheduler/streams/{id}</code> .                            |
| Генерация       | <code>POST /scheduler/generate</code> ; <code>GET /scheduler/tasks</code> ; <code>POST cancel/retry/publish/archive</code> ; <code>diagnostics/components</code> . |
| Расписание      | <code>GET /scheduler/schedule</code> ; <code>PATCH/DELETE /scheduler/schedule/{entry_id}</code> ; <code>GET /scheduler/export</code> .                             |
| Планирование    | Создание периода, список недель, распределение и замена недельной нагрузки.                                                                                        |
| Справочники     | CRUD для аудиторий и справочных сущностей, доступности, профилей правил; обновление преподавателя, группы и требований потока.                                     |

Длительный расчёт создаёт `GenerationTask` и обрабатывается `worker`. Блокировки области расчёта не позволяют одновременно запускать пересекающиеся задачи. При отмене отзываются задачи Celery; при потере `worker` задача переводится в `failed`, сохраняется диагностика, а блокировки освобождаются.

## 8. Проверка, мониторинг и резервное копирование

| Проверка        | Команда / критерий                                                                                                                |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Backend-стиль   | <code>ruff check src/app</code>                                                                                                   |
| Backend-тесты   | <code>python -m pytest</code>                                                                                                     |
| Frontend        | <code>cd spa &amp;&amp; npm run build</code>                                                                                      |
| Схема           | <code>cd src &amp;&amp; alembic upgrade head &amp;&amp; alembic check</code>                                                      |
| Compose         | <code>docker compose --env-file .env.example config --quiet</code>                                                                |
| Здоровье        | GET /health/live — процесс; GET /health/ready — соединение с БД.                                                                  |
| Резервная копия | Регулярный <code>pg_dump</code> БД; отдельно сохранять том <code>uploads</code> и, при необходимости, <code>backend_logs</code> . |

**Приёмка обновления.** После развёртывания проверьте параллельный запуск, отмену расчёта, повторный запуск, конфликтное перемещение записи, фиксацию занятия, публикацию новой версии и Excel-экспорт опубликованного расписания.